

Interactive CPU Scheduling Simulator with AI-Powered Analysis: A Web-Based Educational Platform for Operating Systems

1st Tejmul Movin

Dept. of Computer Science
Rishihood University, Sonipat, India
sansar.t23csai@nst.rishihood.edu.in

2nd Akash G

Dept. of Computer Science
Rishihood University, Sonipat, India
akash.g23csai@nst.rishihood.edu.in

3rd Bhavani Shankar

Dept. of Computer Science
Rishihood University, Sonipat, India
pichuka.b23csai@nst.rishihood.edu.in

Abstract—CPU scheduling is a fundamental operating systems concept that determines process allocation and execution order. This paper presents an interactive web-based CPU Scheduling Simulator (Team 404) built with Next.js 14, React 18, and TypeScript. The platform implements seven scheduling algorithms—FCFS, SJF, SRTF, Priority, Round Robin (RR), Multilevel Queue (MLQ), and Multilevel Feedback Queue (MLFQ)—with real-time Gantt-chart visualization and side-by-side algorithm benchmarking. A key innovation is the integration of two AI-powered features: (1) an *AI Performance Insights Panel* powered by Google Gemini that automatically analyzes simulation results and identifies convoy effects, starvation, and context-switch overhead; and (2) an *AI Process Generator* powered by Anthropic Claude (SchedBot) that creates pedagogically appropriate process sets from plain-English descriptions. Together these features bridge the gap between raw scheduling metrics and deep conceptual understanding, making the platform highly effective for operating systems education.

Index Terms—CPU Scheduling, Operating Systems, Educational Software, Interactive Visualization, AI Assistant, FCFS, SJF, Round Robin, Multilevel Feedback Queue, Large Language Models

I. INTRODUCTION

CPU scheduling is critical to operating system performance and fairness. Different scheduling algorithms produce significantly different results for the same workload, making empirical understanding essential for computer science students and systems professionals. Traditional textbook approaches rely on static diagrams and worked examples, failing to convey the dynamic interplay of arrival patterns, burst lengths, and algorithm policy.

Our CPU Scheduling Simulator addresses this gap with an interactive platform where users can define processes with custom parameters, execute any of seven scheduling algorithms, and observe execution through animated Gantt charts and live metrics. Two AI-powered features—automatic performance analysis and natural-language process generation—further lower the barrier to understanding.

The simulator is deployed at cpu-scheduling-navy.vercel.app and its source is available at github.com/akashg7/CPU-Scheduling.

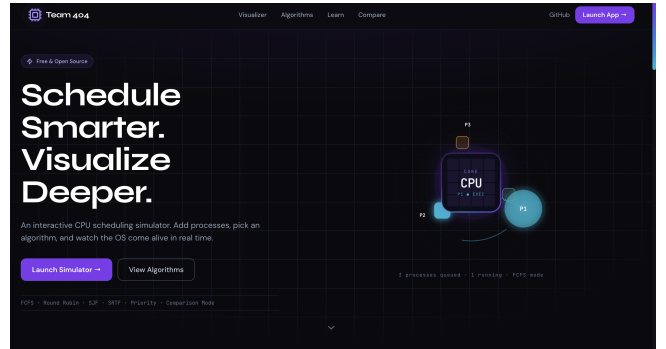


Fig. 1. Landing page hero: “Schedule Smarter. Visualize Deeper.” with an animated CPU visualizer showing three orbiting processes. The platform is freely accessible at cpu-scheduling-navy.vercel.app.

II. SYSTEM ARCHITECTURE

A. Technology Stack

The application is built on a modern, type-safe stack. **Next.js 14** (App Router) with **React 18** provides server-side rendering and component-based UI. **TypeScript** enforces end-to-end type safety. **Tailwind CSS** delivers a fully responsive dark-themed design system using a custom `schedos` palette. **Zustand** manages global state with minimal boilerplate. **Framer Motion** drives smooth Gantt animations and UI transitions. The AI layer uses **Anthropic Claude** (claude-sonnet) via a direct API for the SchedBot chat interface and **Google Gemini** (gemini-2.5-flash) via a separate serverless route for automated performance insights. Deployment is on **Vercel** with automatic GitHub-triggered builds.

B. Application Routes

Three primary routes implement the platform: the landing page (`/`), the main simulator and comparison view (`/visualizer`), and the algorithm reference guide (`/guide`). A centralized Zustand store (`store/useStore.ts`) exposes actions for process management, algorithm selection, simulation execution, and play-

back control. Each scheduling algorithm is an independent TypeScript module under `lib/algorithms/` returning a `SimulationResult` that contains the full Gantt chart, per-process metrics, ordered system snapshots, and typed event logs.

III. IMPLEMENTED SCHEDULING ALGORITHMS

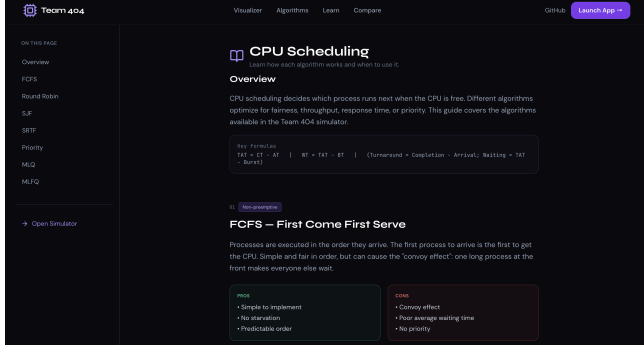


Fig. 2. Algorithm Guide page documenting each scheduling algorithm with key formulas, pros/cons cards, and direct “Open Simulator” links for hands-on exploration.

A. FCFS and SJF

First Come First Served (FCFS) executes processes in arrival order—simple but prone to the convoy effect when long processes precede short ones. Shortest Job First (SJF) selects the minimum-burst process from the ready queue, minimizing average waiting time for non-preemptive scenarios.

B. SRTF and Priority

Shortest Remaining Time First (SRTF) preempts the running process whenever a newly arrived process has a shorter remaining burst, improving average response time. Priority Scheduling uses assigned priority numbers (lower = higher priority) with arrival time as tie-breaker; the current implementation is non-preemptive.

C. Round Robin

Round Robin allocates a configurable time quantum to each process. Processes that do not finish within their quantum are re-enqueued, ensuring fairness and preventing starvation at the cost of higher average turnaround time.

D. MLQ and MLFQ

Multilevel Queue (MLQ) partitions processes into three static queues: Q1 (System, RR), Q2 (Interactive, RR), Q3 (Batch, FCFS), with higher queues preempting lower ones. Multilevel Feedback Queue (MLFQ) adds dynamic queue movement: aging promotes long-waiting processes upward (Q3→Q2→Q1) while feedback demotes processes that exhaust their quantum downward (Q1→Q2→Q3), enabling adaptive priority management without static assignments.

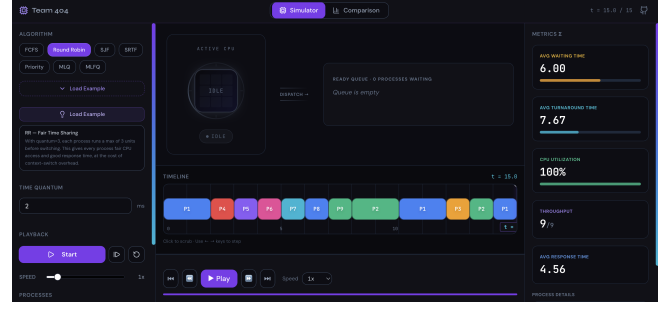


Fig. 3. Simulator view with Round Robin (quantum = 2) on 9 processes. Left panel: algorithm selector, time-quantum input, and Load Example control. Center: color-coded Gantt timeline at $t = 15$. Right panel: live metrics—Avg WT 6.00, Avg TAT 7.67, CPU Utilization 100%, Throughput 9/9, Avg Response Time 4.56.

IV. CORE SIMULATOR FEATURES

A. Interactive Simulator Interface

The left configuration panel supports algorithm selection, per-process entry (arrival time, burst time, priority, queue level for MLQ/MLFQ), and a “Load Example” button that populates a curated scenario demonstrating the selected algorithm’s key behavior. Algorithm-specific controls are shown contextually: a time-quantum input for RR; Q1/Q2 quantum values, feedback and aging toggles, and an aging threshold for MLQ/MLFQ.

B. Active CPU and Ready Queue

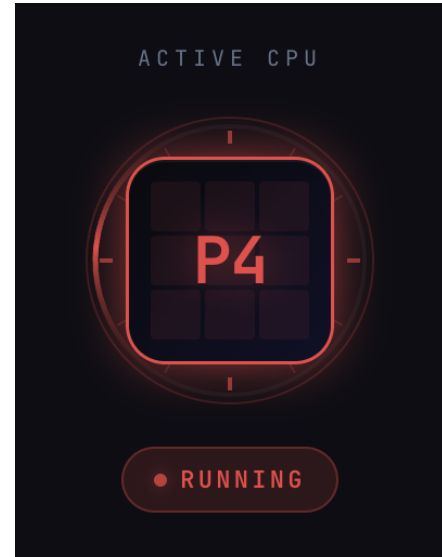


Fig. 4. Active CPU component showing process P4 executing, with a process-colored glowing ring animation and a “RUNNING” status pill that updates in real-time.

The `SimulatorCPU` component renders the currently executing process inside an animated, process-colored glowing ring. The adjacent `ReadyQueue` component lists waiting processes in dispatch order, displaying priority or remaining-burst badges for SJF, SRTF, and Priority algorithms. A context-switch counter derived from simulation logs appears below the queue.

C. Gantt Chart and Playback

The interactive Gantt chart renders color-coded blocks proportional to execution duration with a moving scrubber at the current simulation time. Users click on the timeline to jump to any instant, or use arrow keys for frame-by-frame stepping. Playback speed is adjustable from 0.5× to 5× via a slider, with `requestAnimationFrame` driving time advancement. Framer Motion animates block fills smoothly.

D. Performance Metrics

Metrics are computed continuously as the simulation advances:

$$\text{Waiting Time} = \text{Start Time} - \text{Arrival Time} \quad (1)$$

$$\text{Turnaround Time} = \text{Completion Time} - \text{Arrival Time} \quad (2)$$

$$\text{CPU Utilization} = \frac{\text{Busy Time}}{\text{Current Time}} \times 100\% \quad (3)$$

The metrics panel shows per-process WT, TAT, and completion time together with system-wide averages, CPU utilization, throughput, and average response time, all updating live.

V. ALGORITHM COMPARISON

ALGORITHM	AVG WAITING	AVG TURNAROUND	MAKESPAN
FCFS	2.33	5.33	9
SJF BEST	1.67	4.67	9
SRTF	1.67	4.67	9
Priority	2.33	5.33	9
RR	3.00	6.00	9
MLQ	2.33	5.33	9
MLFQ	2.33	5.33	9

Fig. 5. Comparison tab benchmark. All seven algorithms run simultaneously on the same process set. SJF achieves the best Avg WT (1.67) and Avg TAT (4.67), marked “BEST”. RR produces the highest WT (3.00) and TAT (6.00) for this workload.

The Comparison tab executes all seven algorithms concurrently on the current process set and presents a unified table of average waiting time, average turnaround time, and makespan. The best-performing algorithm is highlighted with a “BEST” badge. Two normalized bar charts accompany the table—one per metric—enabling immediate visual comparison and helping students relate quantitative values to relative bar lengths without switching views.

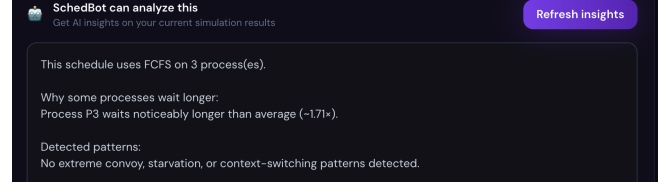


Fig. 6. SchedBot Insights banner embedded in the simulator. After each run, the AI generates a natural-language analysis identifying per-process wait disparities, detected scheduling patterns (convoy, starvation, context-switch overhead), and concrete tuning recommendations.

VI. AI-POWERED FEATURES

A. AI Performance Insights Panel

After a simulation run, the `AskSchedbotBanner` component posts the algorithm, time-quantum, MLQ configuration, process set, and computed metrics to the `/api/insights` serverless route. This route calls **Google Gemini** (`gemini-2.5-flash`) with a structured prompt requesting a six-point analysis:

- 1) Overall schedule summary (algorithm, process count, makespan).
- 2) Which processes waited significantly longer than average and why.
- 3) Detected patterns—convoy effect, starvation, excessive context switches.
- 4) Concrete parameter-tuning recommendations, e.g., “reducing the time quantum from 5 to 3 would lower average response time by ≈20%”.
- 5) Comparative suggestion if a different algorithm would perform materially better.
- 6) One-line conclusion.

When the Gemini API is unavailable the route falls back to a deterministic heuristic analyzer applying threshold-based rules to the same metrics. A five-minute in-memory cache keyed by MD5 of the request body prevents redundant API calls during repeated runs.

B. AI Process Generator (SchedBot)

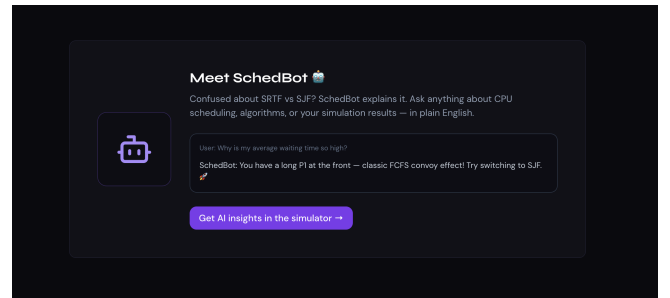


Fig. 7. SchedBot landing section. The conversational interface accepts natural-language scenario descriptions—e.g., “Show me starvation in priority scheduling”—and returns a ready-to-simulate process set with pre-configured arrival times, burst times, and priorities.

SchedBot is a full conversational AI assistant powered by **Anthropic Claude** (`claude-sonnet`) through the `/api/chat` serverless route. Beyond answering conceptual

questions about scheduling algorithms, SchedBot implements an *AI Process Generator*: users describe a scheduling phenomenon in plain English—for example, “Give me a set of processes that demonstrates the convoy effect in FCFS” or “Create a workload that causes starvation under priority scheduling”—and SchedBot generates a complete, pedagogically appropriate process table with arrival times, burst times, and priorities pre-configured to exhibit that behavior. This eliminates the manual trial-and-error of constructing illustrative examples and is especially valuable in classroom and self-study settings.

When a user navigates from the simulator to SchedBot, the current simulation state is automatically passed via `sessionStorage`. SchedBot immediately generates a contextual analysis of those specific results before opening the chat for follow-up questions, creating a seamless handoff between simulation and AI-powered explanation.

VII. EDUCATIONAL BENEFITS

Students can instantly test hypotheses by modifying process parameters and observing metric changes, building intuition about why SJF outperforms FCFS on a given workload. The Comparison tab delivers quantitative trade-off analysis in a single unified view, while the Guide page provides structured documentation with pros, cons, and direct simulation links.

SchedBot’s natural-language explanations lower the barrier for students who struggle to interpret raw metrics. Identified patterns—such as “P3 waits $1.71\times$ longer than average”—create immediately actionable learning moments. The AI Process Generator allows instructors to request demonstrations of specific phenomena such as starvation, convoy effects, or preemption behavior, without pre-computing examples. Students can likewise use SchedBot to generate and then explain any scheduling scenario during self-study, creating a tight prediction-simulation-understanding feedback loop.

VIII. LIMITATIONS AND FUTURE WORK

The current implementation handles single-processor, CPU-bound workloads on discrete integer time units only. Multi-processor scheduling, I/O burst simulation, and fractional time support are planned. The Priority algorithm is implemented as non-preemptive; a preemptive variant will be added. The `/schedbot` page route is not yet registered in the App Router, and the SchedBotFloat button is not yet mounted in the global layout.

Future enhancements include real-time scheduling algorithms (Rate Monotonic, Earliest Deadline First), a quiz mode for predicting behavior before simulation, export of scenarios and metrics to PDF/CSV, and session analytics for tracking learning progress.

IX. CONCLUSION

The Interactive CPU Scheduling Simulator demonstrates how modern web technologies and large language models can together create highly effective educational tools for

complex operating systems concepts. Seven scheduling algorithms with real-time animated Gantt visualization, side-by-side benchmarking, AI-generated natural-language insights, and an AI Process Generator that converts plain-English descriptions into simulation-ready process sets collectively lower the barrier to understanding CPU scheduling.

Interactive visualization makes scheduling dynamic rather than abstract; direct algorithm comparison reveals trade-offs quantitatively; and AI-powered analysis bridges the gap between raw metrics and conceptual understanding. The modular architecture enables straightforward future expansion. The simulator is publicly accessible at cpu-scheduling-navy.vercel.app, providing immediate value to students, educators, and practitioners.

ACKNOWLEDGMENT

We acknowledge the support of Rishihood University’s Department of Computer Science and our instructors for their guidance. Special thanks to the open-source communities behind Next.js, React, TypeScript, Tailwind CSS, Zustand, and Framer Motion, and to Anthropic and Google for the AI APIs powering SchedBot and the insights engine.

REFERENCES

- [1] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Pearson, 2014, pp. 195–268.
- [2] G. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Wiley, 2018.
- [3] A. Medina, A. Banerjee, and I. Constantinides, “Scheduling algorithms in operating systems,” *J. Computing Science and Engineering*, vol. 8, no. 2, pp. 112–125, 2014.
- [4] R. Mayer, “The promise of multimedia learning,” *Learning and Instruction*, vol. 13, no. 2, pp. 125–139, 2003.
- [5] C. Ruhland, “Benefits of interactive visualization in computer science education,” *IEEE Trans. Education*, vol. 49, no. 3, pp. 363–369, 2006.
- [6] A. Verma and S. Khurana, “Performance analysis of CPU scheduling algorithms,” *Int. J. Engineering Research and Applications*, vol. 4, no. 5, pp. 124–130, 2014.
- [7] T. Brown *et al.*, “Language models are few-shot learners,” in *Proc. NeurIPS*, vol. 33, pp. 1877–1901, 2020.